CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 7: Queue Implementation using Array and Linked List, with Structure Simulation

Practical Aim :

To implement queue using array and linked list, and apply queue-based logic to number generation, and structure simulation problems.

Problem 1 :

A government office has a token counter that issues tokens to visitors. The counter can hold at

most n tokens at a time. New visitors join from one end and are served from the other end in the

order they arrived. If the counter is full, no new token can be issued, and if it is empty, no one

can be served. Given a sequence of join and serve operations, implement this fixed-capacity

token system and print the current front token after each operation. Report an error if an

operation cannot be performed.

Describe how your solution tracks the front and rear of the queue. What happens to your front

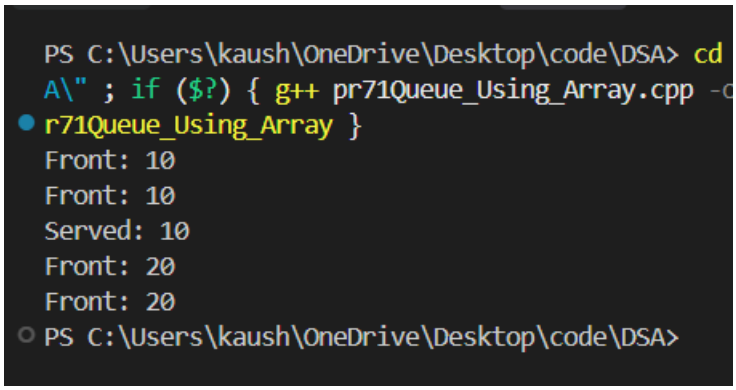
and rear pointers over time as join and serve operations keep happening, and does your

solution handle the case where space appears to run out even though slots have been freed?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/main/FDSAGITSUBMITION

Output:



Problem Faced:

Understanding the top pointer and handling stack overflow and underflow conditions was the main challenge.

Outcome/Conclusion:

The stack was successfully implemented using an array. Push and pop operations were performed correctly with proper full and empty checks.

Problem 2 :

A hospital emergency ward receives patients continuously and attends to them strictly in the

order they arrived, with no upper limit on how many can be waiting at once. New patients are

added at the back and the doctor always attends to the patient at the front. Given a sequence of

arrive and attend operations, implement this unlimited patient queue and print the current front

patient after each operation.

Describe how your solution differs from Problem 1 in how it manages capacity. What happens in your solution when the attend operation is called on an empty ward with no patients waiting?

For Code:

https://github.com/KaushalPatel22/FDSA/tree/7b757e98950f9003ab582799d5dc37f284b72f8e/FDSAGITSUBMITION

Output:

```
PS C:\Users\kaush\OneDrive\Desktop\code\DSA> cd
A\" ; if ($?) { g++ pr72Queue_Using_Linked_List.
f ($?) { .\pr72Queue_Using_Linked_List }
Front: 10
Front: 10
Front: 10
Served: 10
Front: 20
Served: 20
Front: 30
PS C:\Users\kaush\OneDrive\Desktop\code\DSA>
```

Problem Faced:

Managing front and rear pointers and understanding dynamic memory allocation.

Outcome/Conclusion:

Successfully implemented queue operations using arrays and linked lists.

Key Questions / Analysis / Interpretation to be Evaluated During/After Implementation

1. [Problem 1] A simple array queue eventually reports being full even when slots have been freed at the front. Explain why this happens structurally — not by tracing an example, but by describing what the front and rear pointers represent and what their movement pattern causes over time.

ANS :

In an array queue, front moves forward during deletion, but rear does not move backward. So, the queue becomes full even when front spaces are empty.

2. [Problem 2] Your linked list queue has no capacity limit. If arrive operations vastly outnumber attend operations over a long period, what real consequence would this have on the system, and where in your code is that consequence invisible?

ANS :

If arrivals exceed attendances, the queue keeps growing and uses more memory. This is not visible because the code has no memory limit check.